

O'REILLY®

Snowflake architecture

Intro into Snowflake
platform

Tomas Sobotik



Tomas Sobotik

<https://www.linkedin.com/in/tomas-sobotik/>

- Based in Czech Republic
- 2 kids
- 15+ years in data industry
- Data engineer and architect
- Big Snowflake fan and enthusiast
- Snowflake Data Superhero
- Snowflake Subject Matter Expert





Related courses

- **SnowPro Core Video Course:**
 - 5 hours of video content, 65 questions
 - <https://learning.oreilly.com/course/snowpro-core-certification/0642572104672/>
- **SnowPro Core Live Course:**
 - Prep for SnowPro Core Certifications
- **Optimizing Snowflake Live Course**
 - Snowflake performance tuning
- **Data Engineering Bootcamp for Snowflake – Live Course**
 - Comprehensive introduction into data engineering in Snowflake
- **Administering Snowflake – Live Course**
 - How to administer Snowflake



Snowflake in a nutshell

- Founded in 2012 and publicly launched in 2014
- Software as a Service (SaaS)
 - No license to buy, no HW to maintain
- Originally with DWH in the cloud workload
- Only public cloud (AWS, Azure, GCP)
- Separated storage and compute
 - Possibility to scale each part independently
- Per second billing (pay as you go)



ARCHITECTURE

Platform requirements



Fast for any Workload



Run any number or type of job across all users and data volumes quickly and reliably

It just works



Replace manual with automated, optimize costs and minimize downtime

Connected to what matters

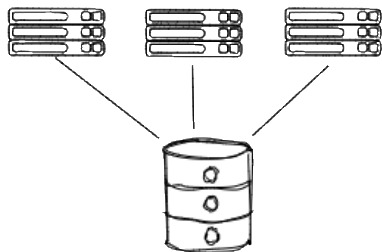


Extend access and collaboration across teams, workloads, clouds - seamlessly and securely

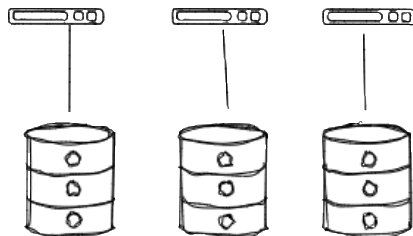
Next level of DWH architecture

Storage and compute separated

Traditional architectures



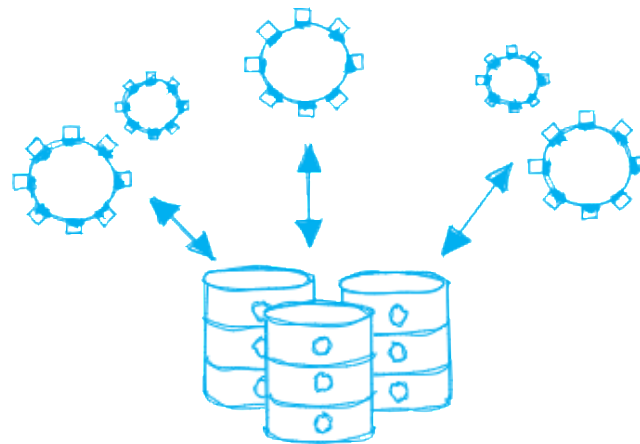
"Shared-disk"
Shared storage
Single Server



"Shared-nothing"
Decentralized, local storage
Single cluster
MPP APPLIANCE

Teradata, Netezza, Vertica
Redshift

Data cloud

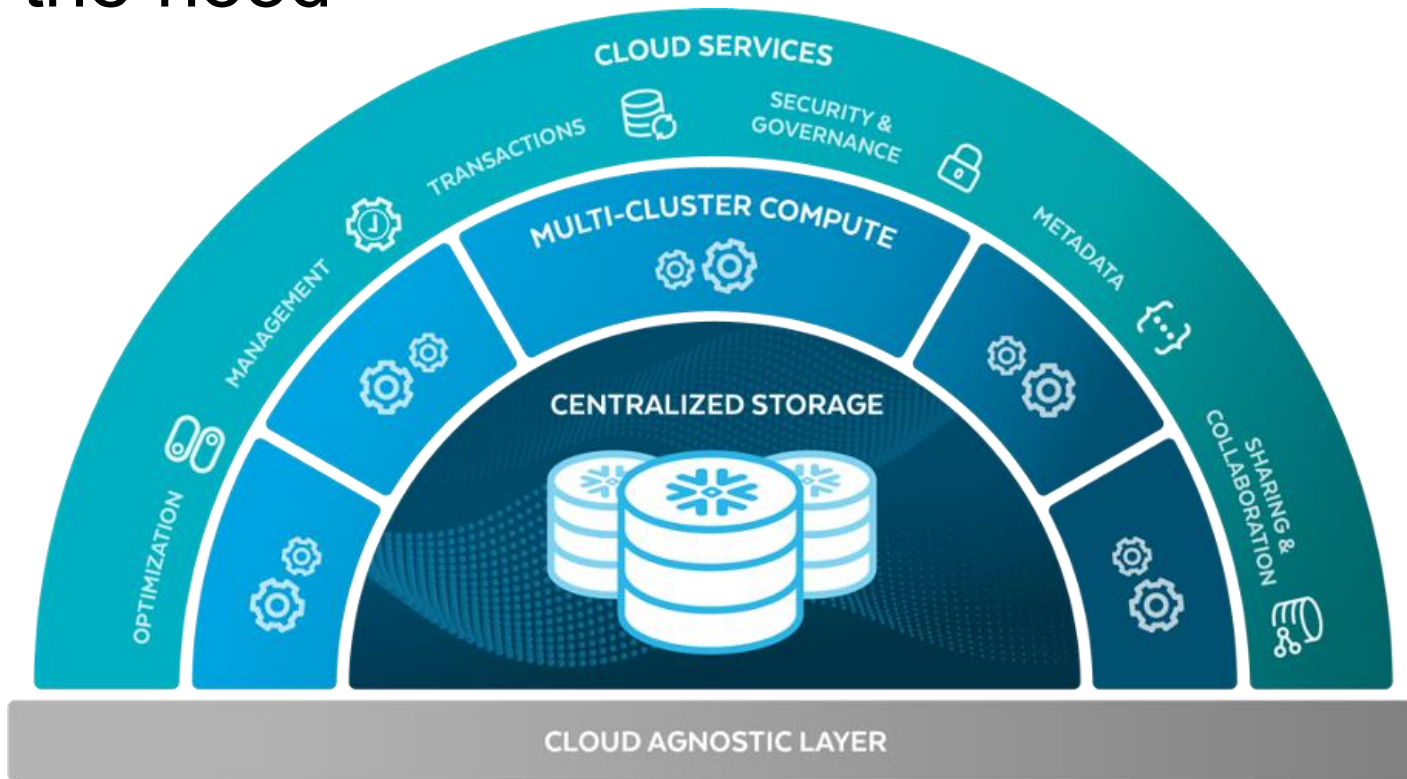


"Multi-cluster, shared data"
Centralized, elastic storage
Multiple MPP, independent compute clusters
UNLIMITED Storage and Compute

Globally



Under the hood



Data Storage



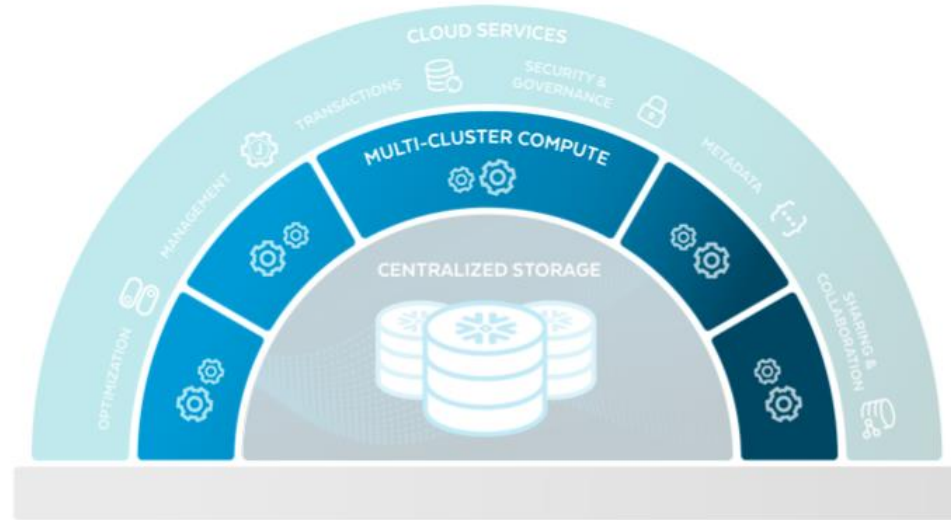
- Centralized storage
- Up to 5x compression
- Native support for semi structured data
- Data are organised in micro partitions
- Zero-copy cloning
- Time travel
- Storage is small part of the overall bill



Compute



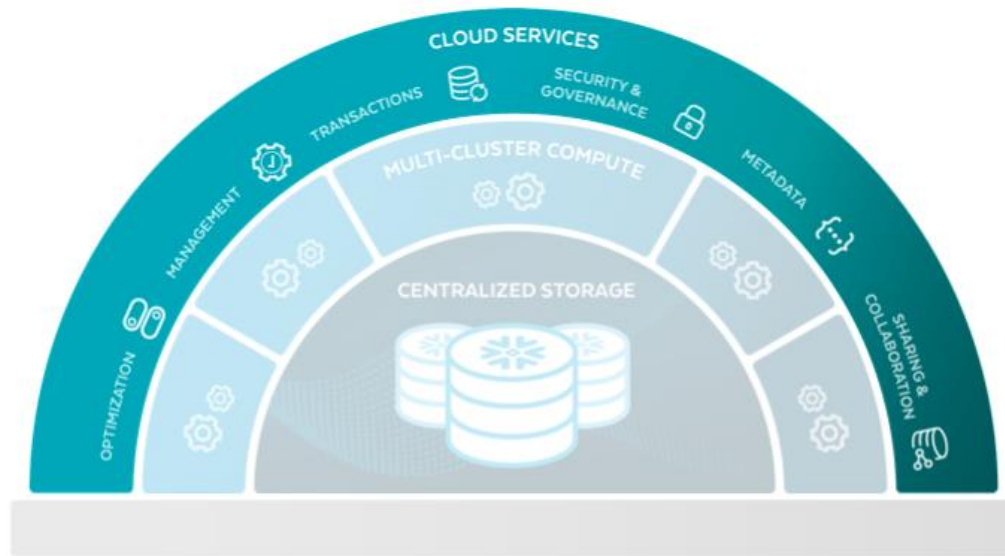
- On-demand virtual warehouses
 - Elastic (turn on/off instantly)
 - Scaling up - bigger
 - T shirt sizes
 - Scaling out - more
 - multi cluster warehouses
- Suspend
- Per second billing
- Most spendings is for virtual warehouses run time



Cloud services

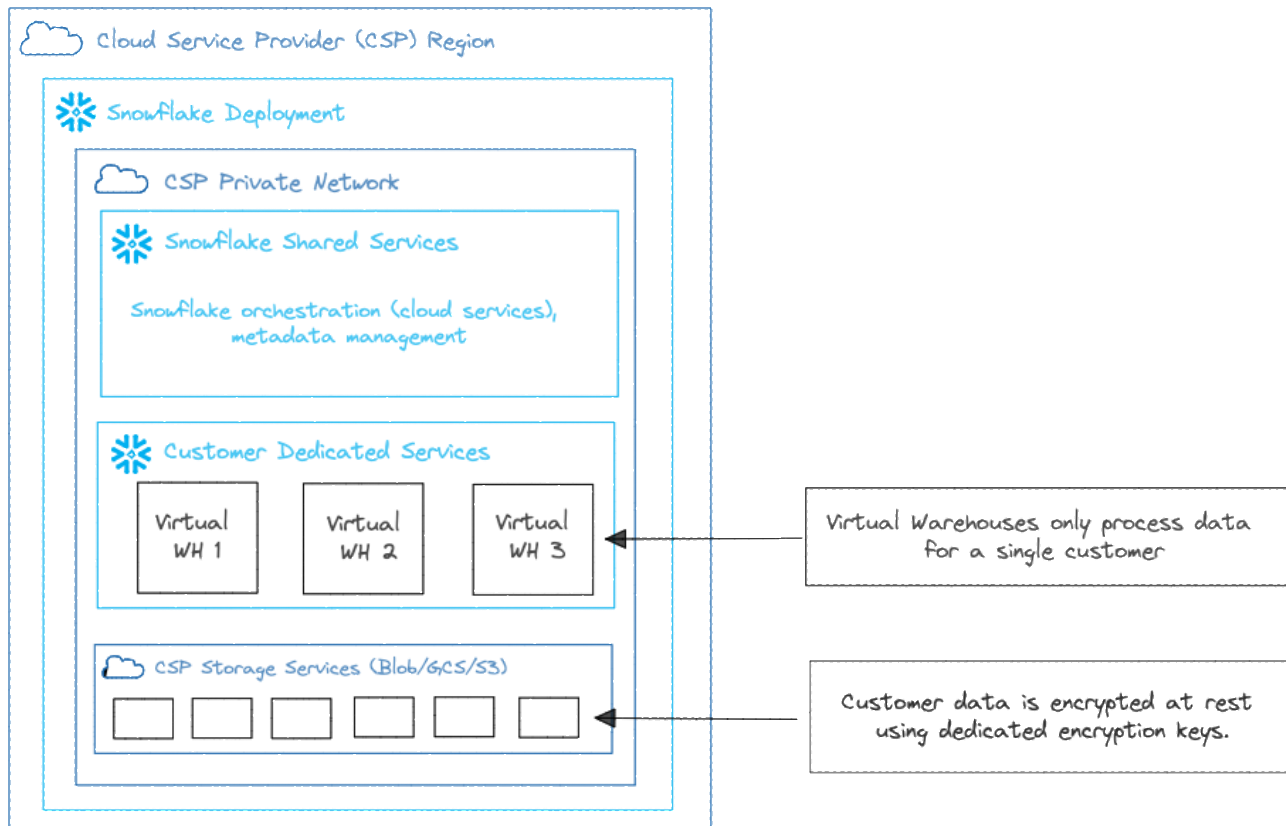


- Compute that does not run on virtual warehouse
 - Show commands
 - Query result cache
- Serverless features
 - Snowpipe
 - Auto-clustering
 - Maintenance of MV
 - Cross - cloud replication





Architecture at a glance



- Customers never have direct access (e.g. SSH) to the Snowflake VPC/VNET
- All access to customer data is through Snowflake Service application layer
- Customer data are decrypted only in memory on dedicated Virtual Warehouse VMs
 - Only the data required to process the command is decrypted
 - Virtual Warehouses are ephemeral, meaning they run only when needed

Poll



1. How many layers has Snowflake architecture?

- a) 2
- b) 3
- c) 4

2. Where can you run Snowflake? - Mark all correct options

- a) Public cloud
- b) Private cloud
- c) Hybrid Cloud
- d) On prem

3. How do you pay for Snowflake? - Mark all correct options

- a) Need to buy a license
- b) Usage based pricing model
- c) Need to buy servers

4. What is Snowflake Storage architecture

- a) Centralized storage
- b) Decentralized Storage
- c) Shared storage

5. Does Snowflake store data compressed?

- a) True
- b) False

6. How many running virtual warehouse can you have?

- a) 3
- b) 5
- c) Almost unlimited

7. Can you change the size of virtual warehouse?

- a) True
- b) False

O'REILLY®

Snowflake key concepts

Basic Snowflake building
blocks – scalability,
separation of workloads

Tomas Sobotik



Snowflake editions



Standard

- Complete SQL Data Warehouse
- Secure Data Sharing across regions / clouds
- Premier support 24/7
- 1 day of time travel
- Always-on encryption in transit and at rest
- Customer dedicated virtual warehouses
- Federated authentication
- Database replication

Enterprise

Standard +

- Multi-Cluster warehouse
- Up to 90 days of time travel
- Annual rekey of all encrypted data
- Materialized views

Business Critical

Enterprise +

- HIPAA support
- PCI compliance
- Data encryption everywhere
- Tri-Secret Secure using customer-managed keys (AWS)
- Enhanced security policy
- Database Failover and Failback for business continuity

Virtual Private Snowflake (VPS)

Business Critical +

- Customer dedicated virtual servers wherever the encryption key is in memory
- Customer dedicated metadata store
- Additional operational visibility

Snowflake Supported Regions



US West (Oregon)
US East (Ohio)
US East (N. Virginia)
Canada Central (Montreal)
US Gov West 1
US East (Commercial Gov – N Virginia)
EU (Ireland)
Europe (London)
EU (Frankfurt)
EU (Stockholm)
Asia Pacific (Tokyo)
Asia Pacific (Osaka)
Asia Pacific (Seoul)
AWS Pacific (Mumbai)
Asia Pacific (Singapore)
Asia Pacific (Sydney)
South American (Sao Paulo)



West US 2 (Washington)
Central US (Iowa)
South Central US (Texas)
East US 2 (Virginia)
Canada Central (Toronto)
US Gov (Virginia)
North Europe (Ireland)
West Europe (Netherlands)
Switzerland North (Zurich)
UAE North (Dubai)
Japan East (Tokyo)
Central India (Pune)
Southeast Asia (Singapore)
Australia East (New South Wales)



US Central 1 (Iowa)
US East 4 (N. Virginia)
Europe West 2 (London)
Europe West 4 (Netherlands)

Intelligent infrastructure



Maintenance
& tuning

Multi-cluster
compute resources

Administration

Networking &
Encryption

Data
Management

Centralized
storage

Automated and fully managed for you

Focus on what matters. Fully managed with automations that encrypt data, control access, and eliminate manual maintenance and troubleshooting.

High availability, high reliability

Automate complex replication and failover cross-clouds and cross-regions. Stay up-and-running no matter what happens.

Optimized costs for all data

Usage-based model paired with patented compression and fine-grained controls to right-size costs. Continual improvements for efficiencies.

Elastic performance engine



One engine for every workload

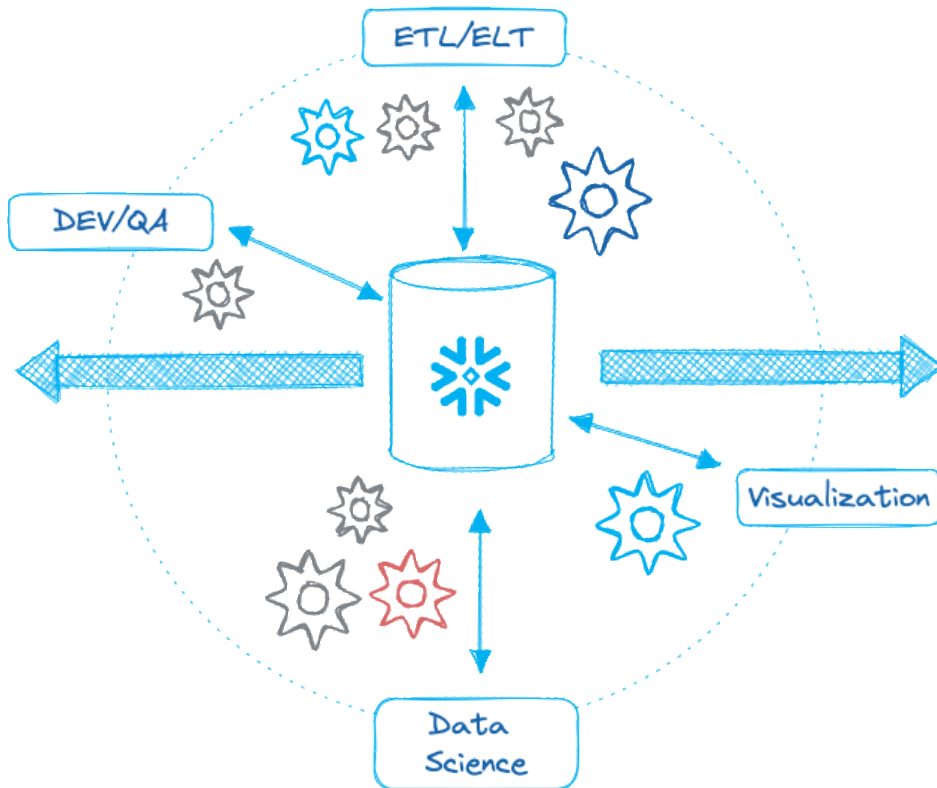
Simplify your architecture. Use dedicated engine per workload and power complex pipelines

Performance and concurrency

No tuning required, instant scaling to almost any number of users, data or jobs

Support any user or skillset

Use SQL, Java, Scala, Python and more. Integrate external tools



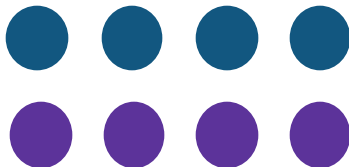
Scaling & elasticity & cost



Same amount of work
in all examples



Thanks to scaling possibilities
the run time is significantly reduce



Cost is same



Time

Supporting many workloads



Collaboration

Cybersecurity

Applications

Data
Engineering

Data
Science

Gen AI

Data
Warehouse

Data
Lake

Unistore

Supporting all data types



Structured Data



Semi-structured Data



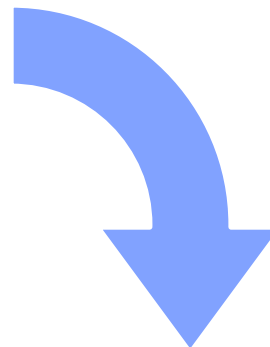
Unstructured Data

Micropartitions

Storage unit holding the data

- Physical files holding the logical tables data
- Many micro partitions per table
- Max 16 MB compressed data ...50 - 500 MB of uncompressed data
- Immutable
 - Updates create a new micropartition
- Snowflake keeps metadata about micro partitions in service layer
 - Helps with query optimization and performance
- Tables divided horizontally in smaller units

ID	NAME
1	Alice
2	Bob
3	Chuck
4	Jane
5	John
6	Mary



P1	ID	1	2	3
	NAME	Alice	Bob	Chuck

P2	ID	4	5	6
	NAME	Jane	John	Mary



Micropartitions metadata

Automatically collected and maintained

- Table level

- Row count
- Table size in bytes
- Referenced files

P1	ID	1	2	3
	NAME	Alice	Bob	Chuck



Micropartition metadata

ID	1-3
NAME	A-C

- Micropartition column level

- Range of value
- Number of distinct values
- Min and MAX values
- NULL count

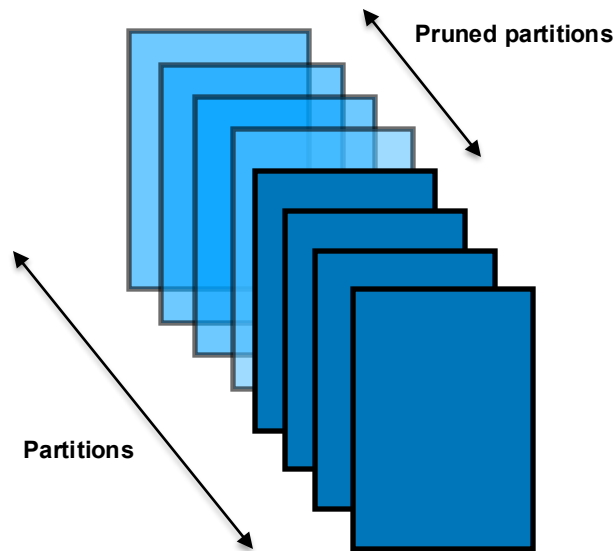
P2	ID	4	4	6
	NAME	Jane	John	Mary



ID	4-6
NAME	J-M

Micropartition advantages

- Metadata used by optimizer for partition pruning
 - Horizontal pruning first - by partition
 - Vertical pruning next - by column
- Unit of management for DML - because of immutability
- Thanks to stored metadata some queries are pure metadata operations
 - No need for running virtual warehouse!
- Data Clustering
 - Effective data loading have impact on query performance



Data stored by column

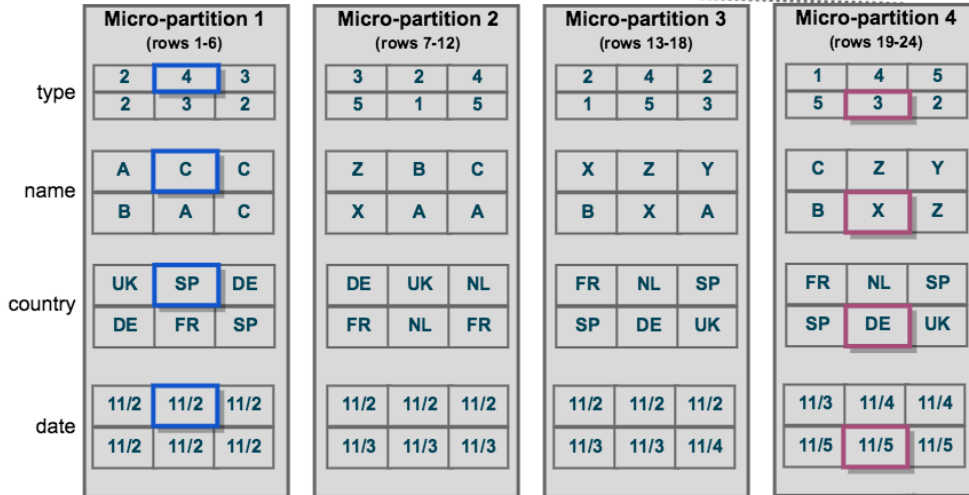


Table: t1

Logical Structure

type	name	country	date
2	A	UK	11/2
4	C	SP	11/2
3	C	DE	11/2
2	B	DE	11/2
3	A	FR	11/2
2	C	SP	11/2
3	Z	DE	11/2
2	B	UK	11/2
4	C	NL	11/2
5	X	FR	11/3
1	A	NL	11/3
5	A	FR	11/3
2	X	FR	11/2
4	Z	NL	11/2
2	Y	SP	11/2
1	B	SP	11/3
5	X	DE	11/3
3	A	UK	11/4
1	C	FR	11/3
4	Z	NL	11/4
5	Y	SP	11/4
5	B	SP	11/5
3	X	DE	11/5
2	Z	UK	11/5

Physical Structure



Poll questions



1. True or False: Snowflake can run in private cloud
 - A. True
 - B. False
2. Which workloads Snowflake supports. Select all correct answers
 - A. Data Warehouse
 - B. Data Apps
 - C. Data Science
 - D. Graph DB
 - E. Data Lake
 - F. Publish - Subscribe service
3. How many compute engines you can have
 - A. 1 in total
 - B. 1 per workload
 - C. Almost unlimited
4. True or false: Snowflake supports semi-structured data
 - A. True
 - B. False
5. Micro partitions could be updated
 - A. Only if there is less than 16 MB of data
 - B. Never
 - C. Whenever it is needed

O'REILLY®

Snowflake UI walkthrough

How to control your
Snowflake account

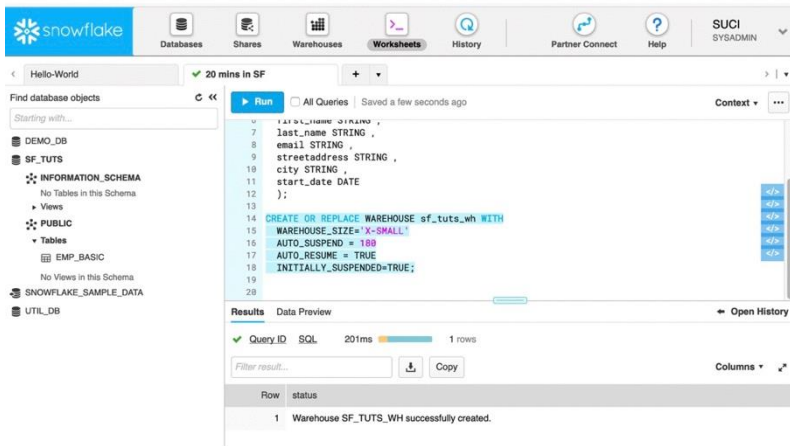
Tomas Sobotik



Snowflake UI tools

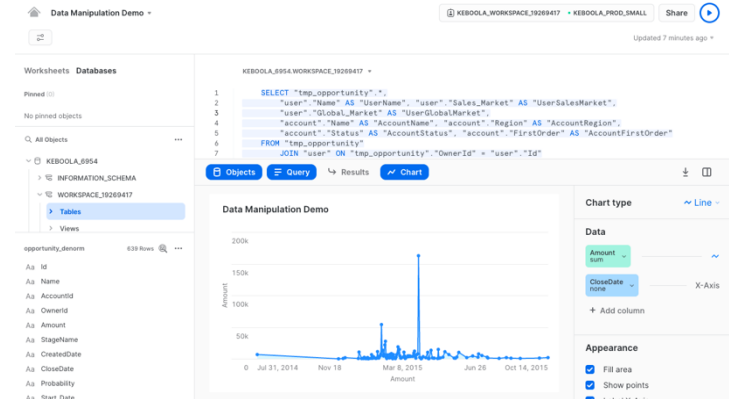
Classic Console

- original Snowflake UI interface
- no new development but still supported
- Will be decommissioned
- no auto complete for a code
- Not available for new accounts



Snowsight

- new Snowflake web interface
- actively developed and improved over time
- modern look and feel
- code auto complete & object names suggestion
- integrated interface for reaching the support
- integrated collaborative features
- integrated query visualization



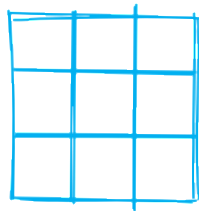
Snowsight demo time



* We are going to work with Snowsight only



Basic tool overview



Using the worksheets



Data
Management



Administration

<https://app.snowflake.com>



DEMO

Exercise



1. Use role `SYSADMIN`
2. Create a database called `CITIBIKE` with **SQL command**
3. Create schema called `WORK`, this time use **Snowsight UI**
4. Create table `TRIPS` in `WORK` schema. Please use **SQL** or **UI** (do not forget to setup the context for your worksheet)

Here you can find columns and data types for the `TRIPS` table:

```
tripduration integer,  
starttime timestamp,  
stoptime timestamp,  
start_station_id integer,  
start_station_name string,  
start_station_latitude float,  
start_station_longitude float,  
end_station_id integer,  
end_station_name string,  
end_station_latitude float,  
end_station_longitude float,  
bikeid integer,  
membership_type string,  
usertype string,  
birth_year integer,  
gender integer
```


Key Takeaways



- DDL operations are free!
- Do not require compute resource



O'REILLY®

Snowflake virtual warehouses

Intro to the Snowflake
compute layer

Tomas Sobotik



Virtual Warehouses



- A named wrapper around a cluster of servers with CPU, memory, and disk
 - A compute cluster = logical collection of VMs (1+)
- The larger the warehouse, the more servers in the cluster
- Snowflake utilize T-shirt sizes for Virtual Warehouses
- Each size up doubles in size
 - Number of nodes correspond to configured size of VW
- Jobs requiring compute run on virtual warehouse
- While running, a virtual warehouse consumes Snowflake credits
- You are charged for compute
- Snowflake account object



Warehouse sizes



Warehouse Size	Servers	Credits / Hours	Credits / Second
X-Small	1	1	0.003
Small	2	2	0.006
Medium	4	4	0.0011
Large	8	8	0.0022
X-Large	16	16	0.0044
2X-Large	32	32	0.0089
3X-Large	64	64	0.0178
4X-Large	128	128	0.0356
5X-Large*	256	256	0.0711
6x-Large*	512	512	0.1422

* Preview feature in some regions



Warehouses operations

- Start / Stop
 - could be automated via warehouse parameters
- AUTO_SUSPEND
 - warehouse will be suspended after specified period of time
 - enabled by default
- AUTO_RESUME
 - warehouse is automatically resumed when any statement that requires a warehouse is submitted
 - current session needs to have specified warehouse
 - enabled by default
- Resizing
 - could be done anytime, even when WH is running and queries are being executed
 - ALTER WAREHOUSE Command or via UI
 - Effects
 - Suspended warehouse will start at new size when it will be resumed
 - Running warehouse: immediate impact
 - running queries - finish with current size
 - new queries run at new size



Instant elasticity

- **Instant provisioning**
 - with few clicks/commands you can enable required amount of computing power based on the size or type of your workload
- **On-demand performance**
 - instantly scale up or down the resource to get required performance
- **On-demand concurrency**
 - multi-cluster warehouse to support your concurrent queries
- **Per-second pricing**
 - predictable cost
- **Faster performance**
 - use large warehouse for shorter periods of time and keep the same cost
- **Reduced cost**
 - idle warehouses could be suspended, pay only for running times



Generation 2 warehouses

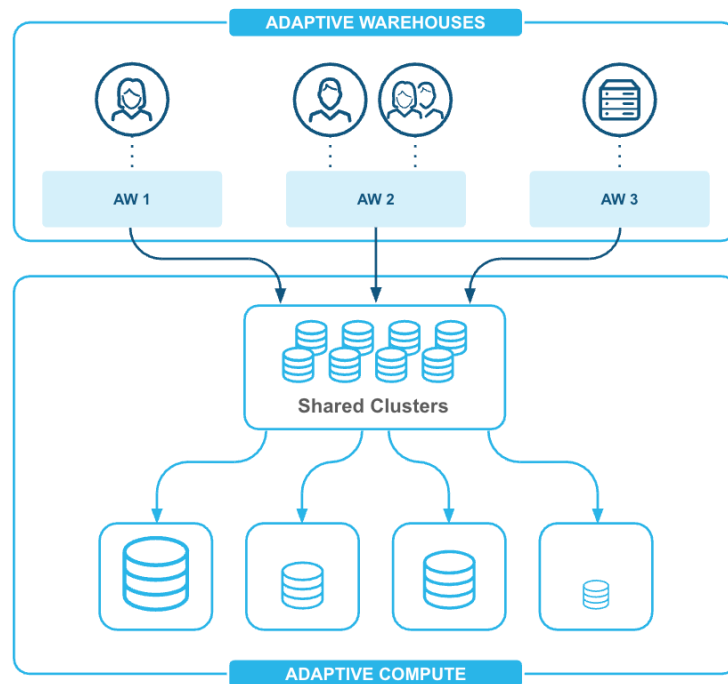
- Updated version of the current standard virtual warehouse
- Newest hardware focused on better performance for analytics and data engineering workloads
- Most of the queries should finish faster
- Limited availability to some some regions only for now
- You can update gen1 to gen2 and other way around
- They are about 30% more expensive than gen1 (1.35 credits for XS size)
- How to create gen2 warehouse

```
CREATE OR REPLACE WAREHOUSE next_gen_wh  
RESOURCE_CONSTRAINT = STANDARD_GEN_2  
WAREHOUSE_SIZE = SMALL;
```



New Adaptive Compute – in preview

- Another simplification
- New warehouse type – Adaptive Warehouse
- Minimal configuration
 - No cluster size selection
 - Automatic suspend/resume
- No need to think about ^{COMPUTE_WH} COMPUTE_WH
 - Size
 - Concurrency settings
 - Multi-cluster settings
- Better price/performance ratio
- Intelligent query routing
- Always running pool of clusters





Scale up for performance

Elastic processing power - CPU, RAM, SSD

Boost the performance for complex queries or ingesting large datasets

More complex queries on larger datasets require larger Warehouse

Concurrency issues (more users/queries) can't be solved by large warehouse

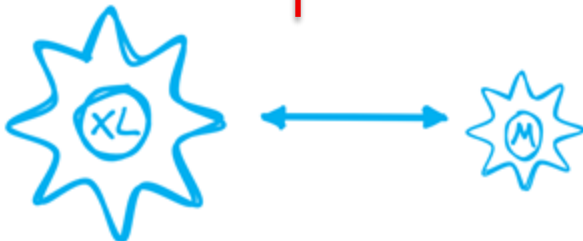
Guidelines for warehouse sizing

Utilize the per second billing

Use larger warehouse for more complex queries, workloads and suspend them when they are not in use

Experiment with warehouse sizes to find the right balance between performance and cost

Keep similar queries on the same warehouse to simplify compute resource sizing





Scale out for concurrency

Multi Cluster Warehouse

- Single virtual warehouse with multiple compute clusters
- Scale out during peak times (more users, concurrent queries) and scale back when additional power is not needed
- Additional resources could be allocated statically or dynamically -> scaling modes & policies
- Queries are distributed across the clusters in a Virtual Warehouse
- To support high availability clusters are deployed across availability zones

Properties

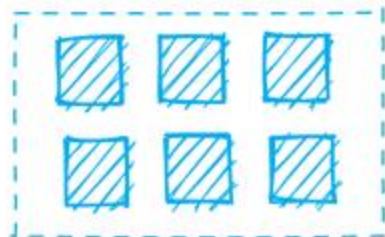
`MIN_CLUSTER_COUNT: 1 - 10 (default 1)`

`MAX_CLUSTER_COUNT: 1 - 10 (default 1)`

`MAX_CLUSTER_COUNT >=`

`MIN_CLUSTER_COUNT`

Multi cluster Virtual Warehouse



Minimum = 6
Maximum = 6



Scaling UP vs. OUT

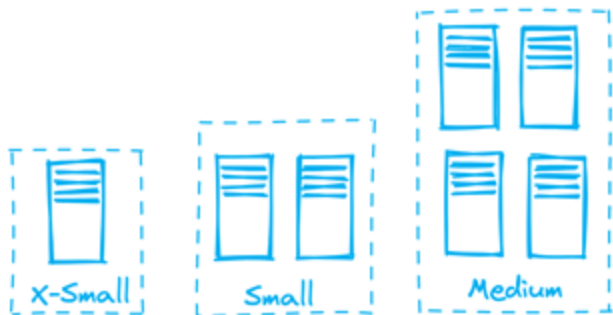
Credits per hour

	1	2	3	4	5	6	7	8	9	10
4XL	128	256	384	512	640	768	896	1024	1152	1280
3XL	64	128	192	256	320	384	448	512	576	640
2XL	32	64	96	128	160	192	224	256	288	320
XL	16	32	48	64	80	96	112	128	144	160
L	8	16	24	32	40	48	56	64	72	80
M	4	8	12	16	20	24	28	32	36	40
S	2	4	6	8	10	12	14	16	18	20
XS	1	2	3	4	5	6	7	8	9	10

Scaling up vs. scaling out II.

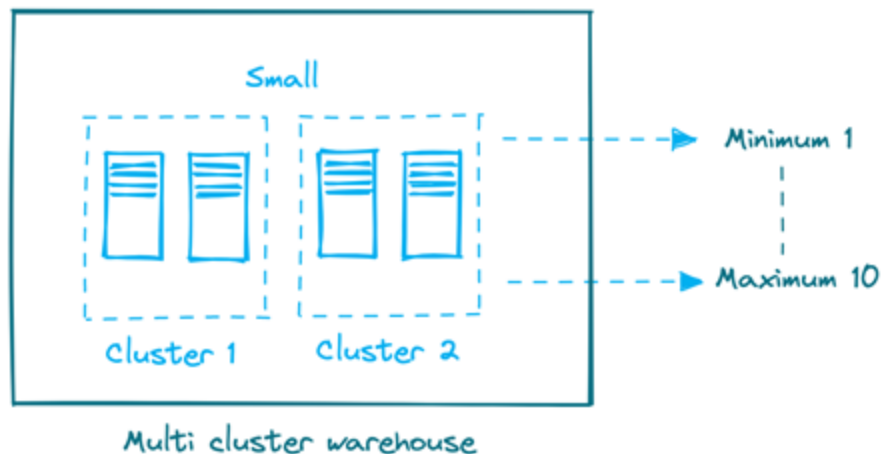
Scaling up

Resizing the single warehouse in order to handle more complex queries with more data



Scaling out

Adding more clusters to the warehouse in order to handle more concurrent requests. Size of the warehouse is not changed.





Scaling modes for multi cluster WH

Maximized

- Static control of compute resources
- Enabled by specifying same value for minimum and maximum number of clusters
 - Value must be larger than 1
- Specified number of clusters is immediately started once warehouse is resumed
- You have large number of concurrent users/queries
- Numbers are rather static and not fluctuate often

Auto-scale

- Dynamic control of compute resources
- Enabled by specifying different values for min and max number of clusters
- Clusters are started/suspended based on the load
- Strategy is controlled by scaling policy
- Load fluctuates over time

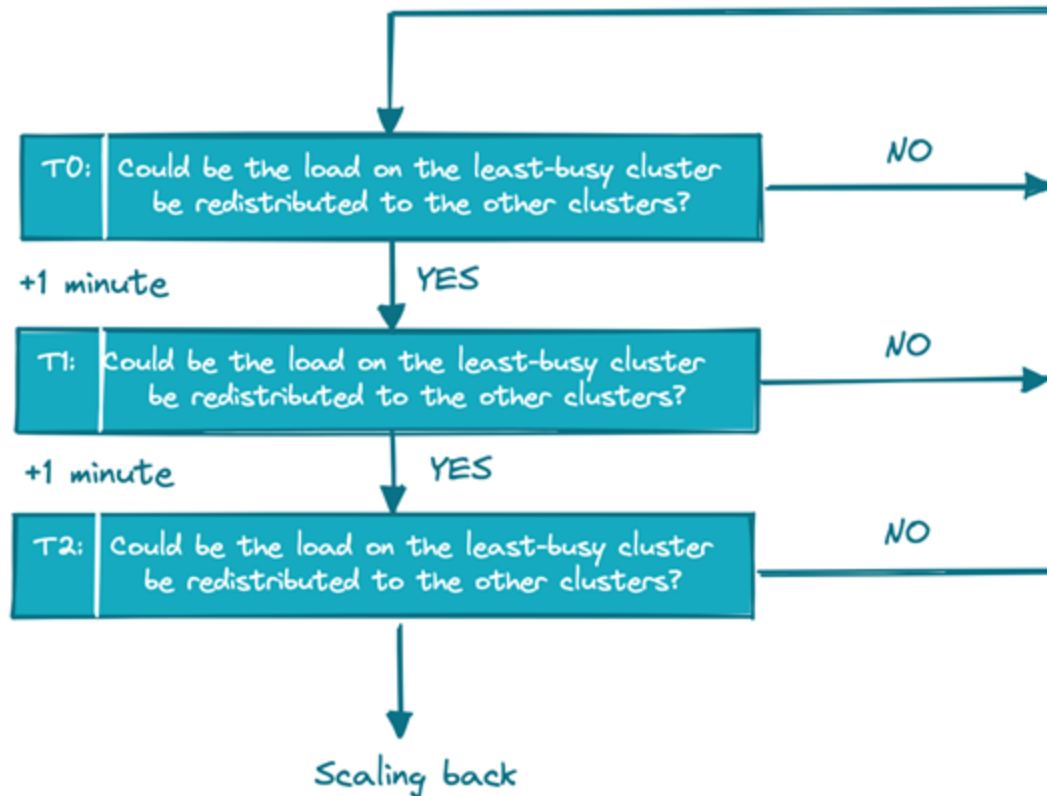
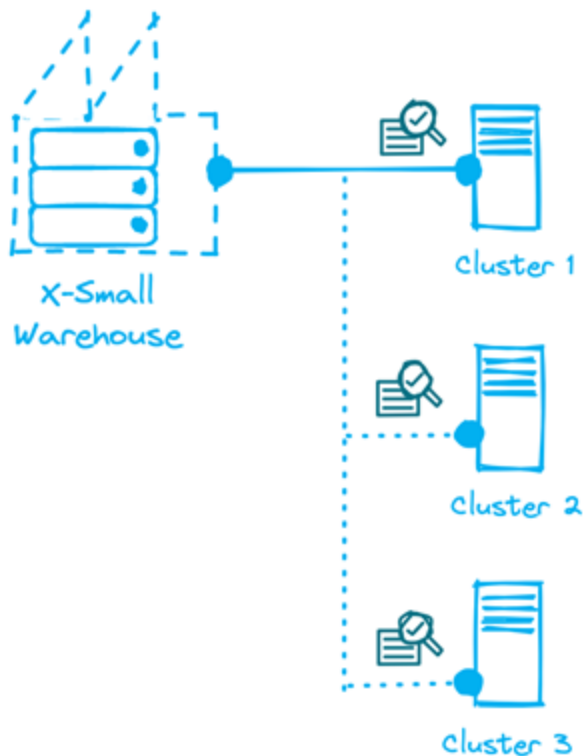


Auto-scaling mode for multi cluster WH

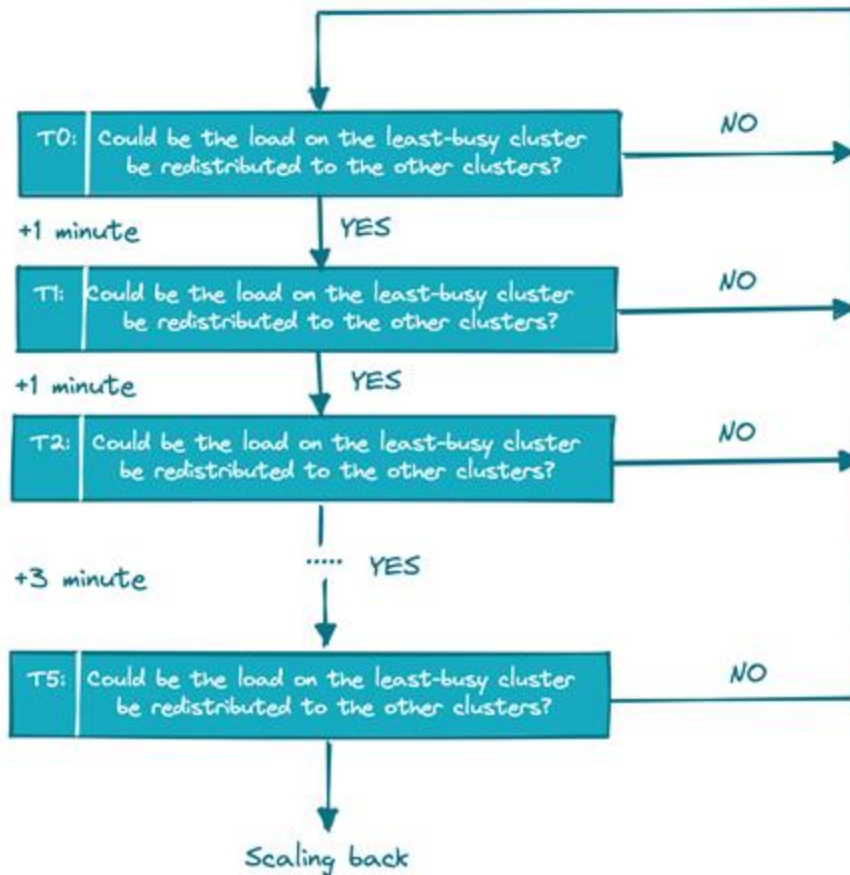
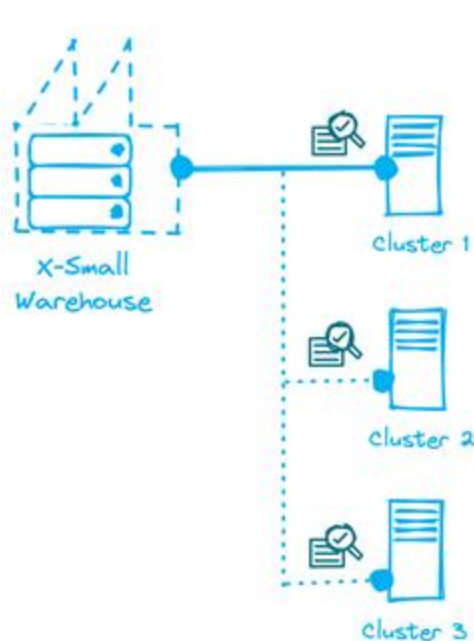
Scaling policy

Policy	Description	cluster starts	cluster shuts down
Standard	Starts a new cluster as soon as queries start to queue	There is queued query or there is more queries than the currently running cluster can execute	After 2-3 consecutive checks (one minute interval) show that the load on the least-busy cluster could be redistributed to other running clusters
Economy	Queries are kept queued up to six minutes - to see if the queue clears. Then another cluster is started	Only if the system estimates there's enough query load to keep the cluster busy for at least six minutes	After 5-6 consecutive checks (one minute interval), show that the load on the least-busy cluster could be redistributed to the others

Standard policy: scaling back workflow



Economy policy: scaling back workflow





Exercise

- Create your first virtual warehouse called `COMPUTE_WH`, with following parameters:
 - Extra small size
 - Enabled auto resume
 - Enabled auto suspend
 - Suspend after 1 minute
 - It will be suspended after creation
- Create a multi cluster warehouse called `MULTI_COMPUTE_WH` with following parameters:
 - Small size
 - Enabled auto resume
 - Enable auto suspend
 - Suspend after 3 minutes
 - It will be suspended after creation
 - Min number of clusters: 1
 - Max number of clusters: 3
 - Economy scaling policy
- Use SQL for virtual warehouse creation
- Use `SYSADMIN` role for object creation
- Use Documentation for the right SQL syntax

O'REILLY®

Billing

How billing works in
Snowflake

Tomas Sobotik



Cost drivers



- SaaS solution - no license or HW to buy
- Usage based pricing
- Billing is done per layer - storage, compute, cloud services
- Base invoiceable unit is called Snowflake Credits
 - Used to pay for the consumption of resources
 - Consumed only when some resource is used
- Consumption model - pay only for what you actually use
 - Storage = TB of disk space
 - Compute = running time of virtual warehouses



Credit price components



Snowflake Edition



Standard, Enterprise,
Business Critical and
Virtual Private Snowflake

Cloud Provider



Snowflake is cloud agnostic,
available on all major clouds

Region



Global presence with
cross region, cross cloud
replication
or data collaboration



Storage costs

- Based on average amount of terabyte storage used per month
- Calculation is based on compressed and ingested data into Snowflake
- Ingested data are always compressed, encrypted and columnarised
 - 5x - 10x compression based on data types
- Calculated as daily average amount
- Storage costs includes
 - Data itself
 - Time Travel Data
 - Fail Safe Data
 - Internal stages



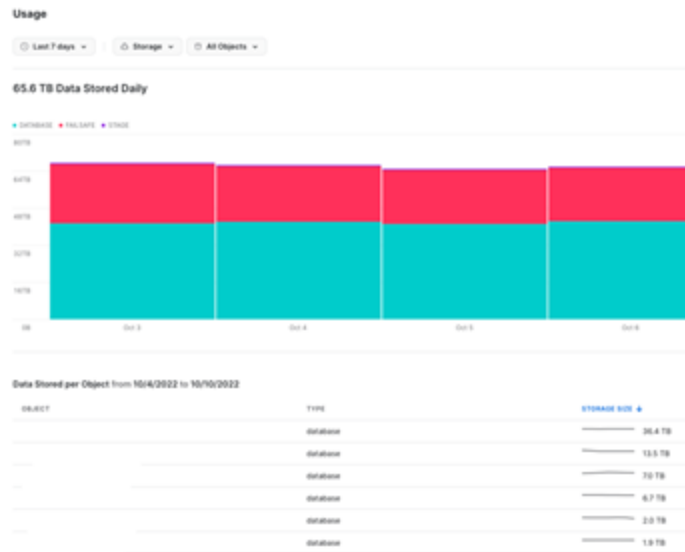


Where to find Data Usage

Snowsight

Admin -> Usage

ACCOUNTADMIN role is needed



SQL

Table functions (Information Schema)

`DATABASE_STORAGE_USAGE_HISTORY`

`STAGE_STORAGE_USAGE_HISTORY`

Views (Account Usage)

`DATABASE_STORAGE_USAGE_HISTORY`

`STAGE_STORAGE_USAGE_HISTORY`

Compute costs



- given by credits consumption
- warehouse size (number of nodes) corresponds to Snowflake credits for full hour of run time
 - XS Warehouse: 1 credit for 1 hour
- charged by second after first minute
- when suspended = no charge (auto suspend!)
- Usually 95% of all costs



	XS	S	M	L	XL	2XL	3XL	4XL	5XL*	6XL*
credits/hour	1	2	4	8	16	32	64	128	256	512



Where to find Warehouse Credit Usage

Snowsight

Admin -> Usage

ACCOUNTADMIN role is needed

SQL

Table functions (Information Schema)

WAREHOUSE_MEERING_HISTORY

Views (Account Usage)

WAREHOUSE_MEERING_HISTOR





Cloud service layer costs

- authentication, metadata management, SQL API, Access Control, Query parsing and optimization
- Snowflake credits are also used for paying the usage of cloud service layer
- Free up to 10% of the daily usage of compute resources

Date	Credits Used	Cloud Services Credits Used	Credit Adjustment for Cloud Services	Credits Billed totally
Feb 21	100	20	-10	110
Feb 22	120	10	-10	120



How to find out cloud service layer usage source

- ACCOUNT_USAGE schema in SNOWFLAKE database
- different perspective
 - query point of view
 - warehouse point of view
- typical use cases
 - find queries which consume the most cloud services credits
 - informative character

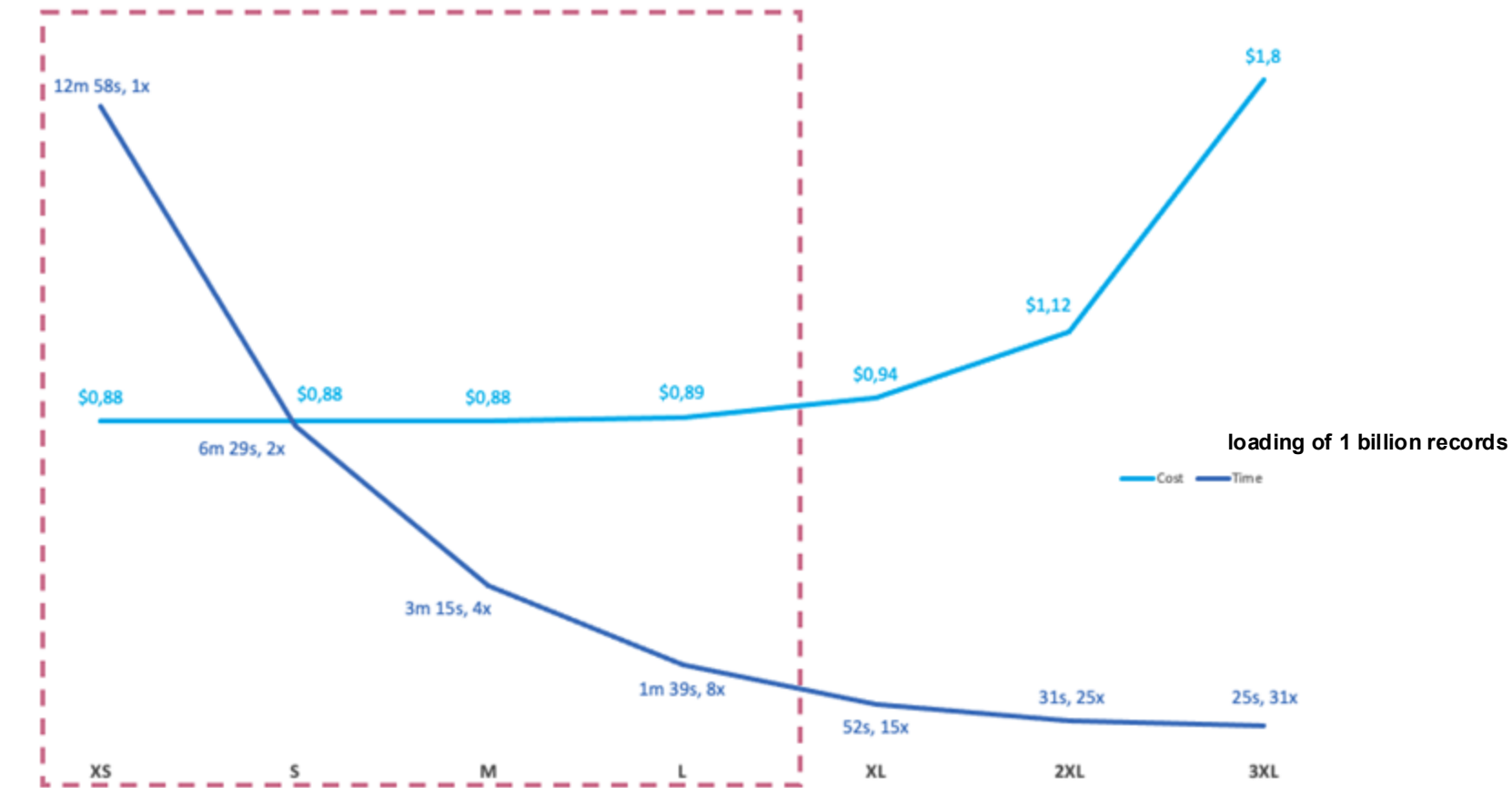


Keep in mind

- some particular features can have significant impact on cost. Needs to be used wisely
 - materialized views
 - auto clustering
 - replication
- always keep cost in mind - it is Pay as you go service
- cost optimization might be important part in later phases of the projects
 - use warehouse parameters
 - utilize Snowflake caches
 - ingest data effectively
 - monitor your costs



Bigger does not mean more expensive





Poll questions

1. How is compute billed in Snowflake
 - A. Fixed amount per month
 - B. Based on current usage
 - C. Compute is free
2. True or false: Storage and compute is billed together
 - A. True
 - B. False
3. What is not Snowflake cost driver
 - A. Number of Snowflake users
 - B. Cloud provider
 - C. Used region
4. True or false: Cloud service layer services are always available for free
 - A. True
 - B. False
5. How can you get the overview about your spendings. Select all correct answers
 - A. Ask Snowflake support
 - B. Admin tab in Snowsight
 - C. Account Usage views

O'REILLY®

Data loading principles

Introduction into DB
objects needed for data
loading

Tomas Sobotik



STAGE

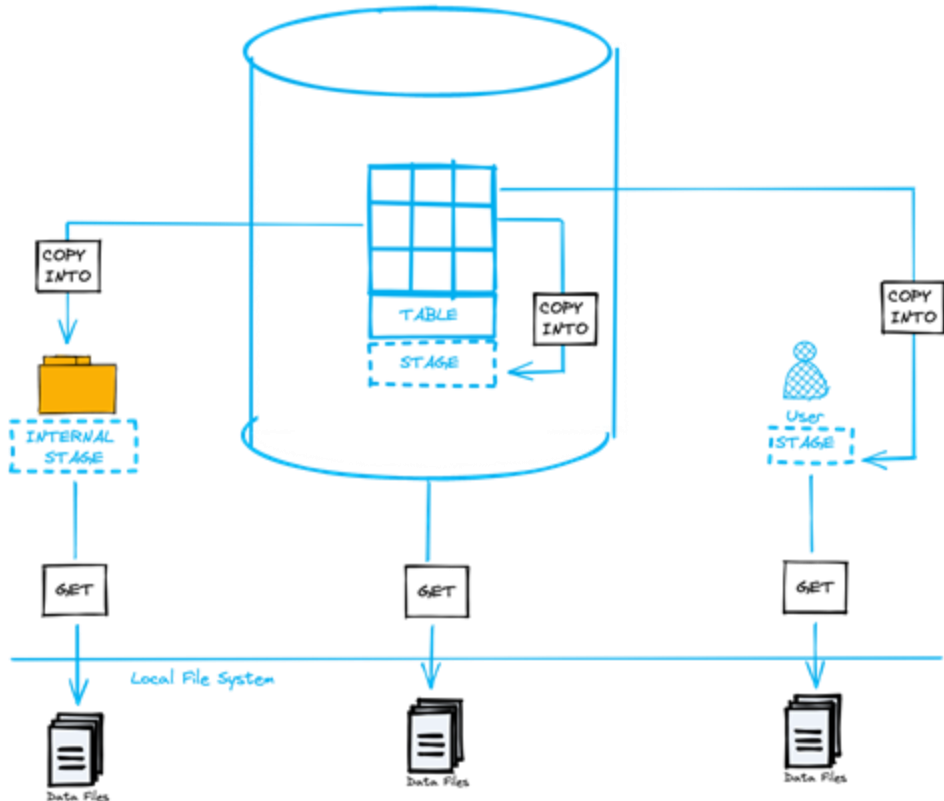
File location description

Internal or External

- Files could be loaded from local machine or existing landing zone in the cloud (S3, Blob storage, Cloud Storage)
- PUT command for loading local files into internal stage

Stage types

- Table Stage: Created automatically for each table: `@%[Table_Name]`
- User stage: Created automatically for each user: `@~`
- Named stage: created manually with SQL - internal or external
- You can't set file format for user and table stages





Basic Stage parameters

- Cloud provider details
 - URL
 - Credentials
 - Can be bundled into special Snowflake object called storage integration = better security
 - Storage integration object can be reused
 - Central place to govern cloud storage access
 - Developers do not need to know cloud storage credentials - access based on IAM roles

```
create storage integration s3_int
  type = external_stage
  storage_provider = 'S3'
  storage_aws_role_arn =
'arn:aws:iam::001234567890:role/myrole'
  enabled = true
  storage_allowed_locations =
('s3://mybucket1/path1/',
's3://mybucket2/path2/');
```

```
create stage my_ext_stagel
  url='s3://load/files/'
  credentials=(aws_key_id='1a2b3c'
aws_secret_key='4x5y6z');
```


FILE FORMAT

File structure description

CSV

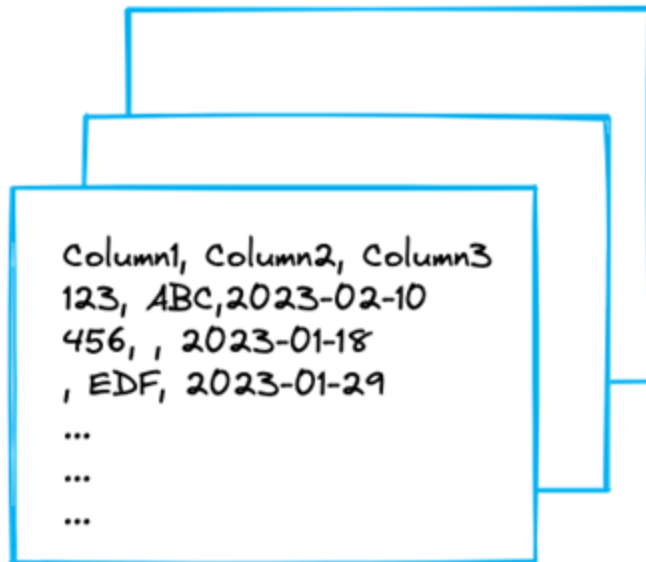
- Record and field delimiters
- Escape characters
- NULL handling and other parameters

JSON

- Strip null values
- Date and time format
- Trim spaces and other parameters

Can be defined on different levels

- STAGE object
- COPY command
- Named object



```
CREATE OR REPLACE FILE FORMAT my_csv
  TYPE = csv
  FIELD_DELIMITER = '\,'
  SKIP_HEADER = 1
  NULL_IF = ''
```



Data loading recommendations



Organize files by path

- Create logical paths per required dimension
 - Time
 - Location
 - Department

`US/east_cost/finance/monthly/2022/02`

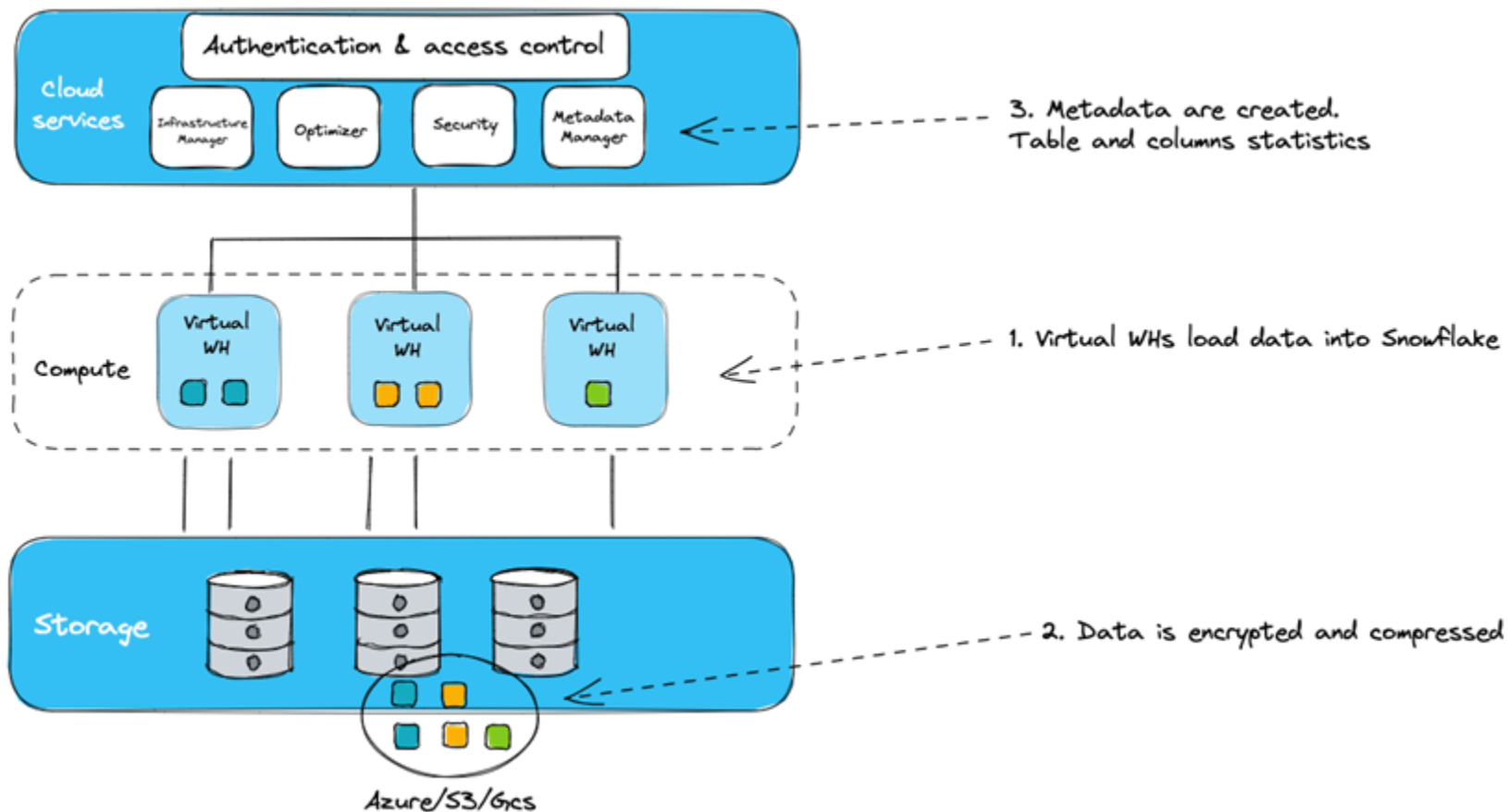
File sizes

- Split large files before loading
- Batch / combine small files before loading
- Recommended size is 100 to 250 MB (compressed)

Warehouse size	Number of Cores	Number of Threads
XS	4	8
S	8	16
M	16	32
L	32	64
XL	64	128

Number of files that
can be loaded in
parallel

Data loading process



File locations



All files in single location

Processing is slower as engine needs to scan more files to find the right ones

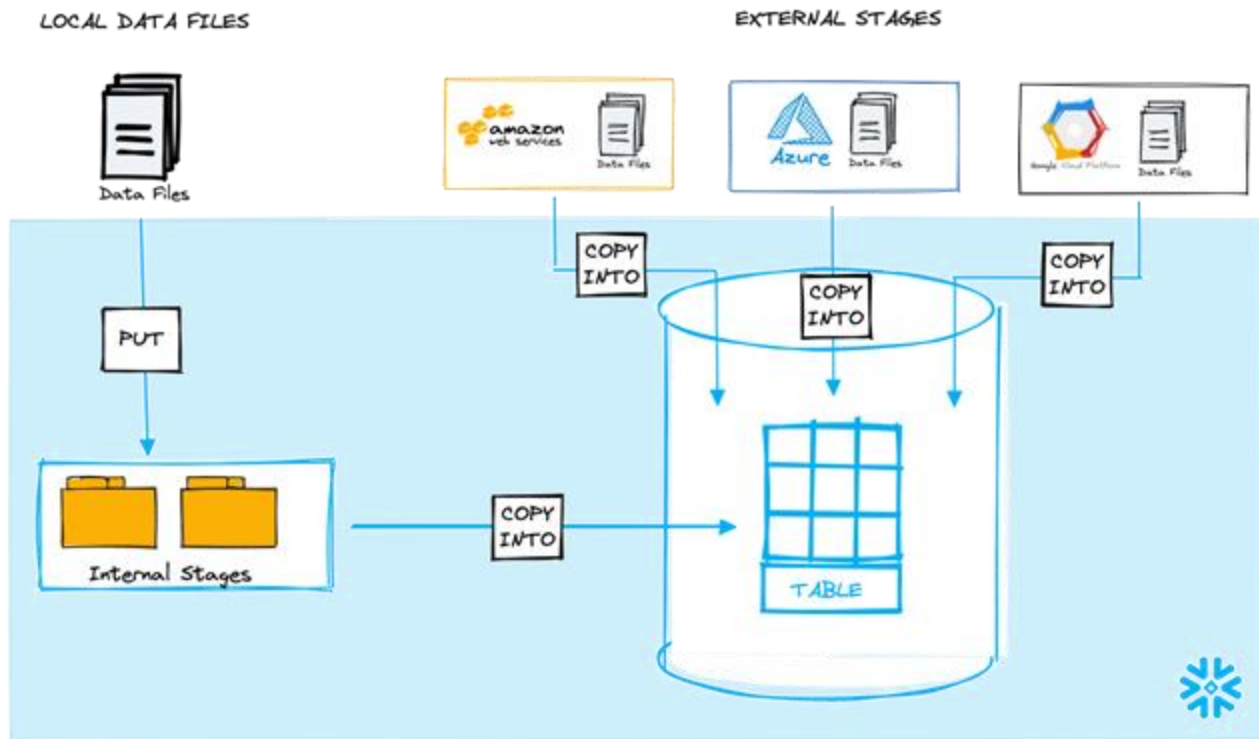


Organised files in multiple locations

Faster. Multiple directories with same file types allow for scanning fewer files



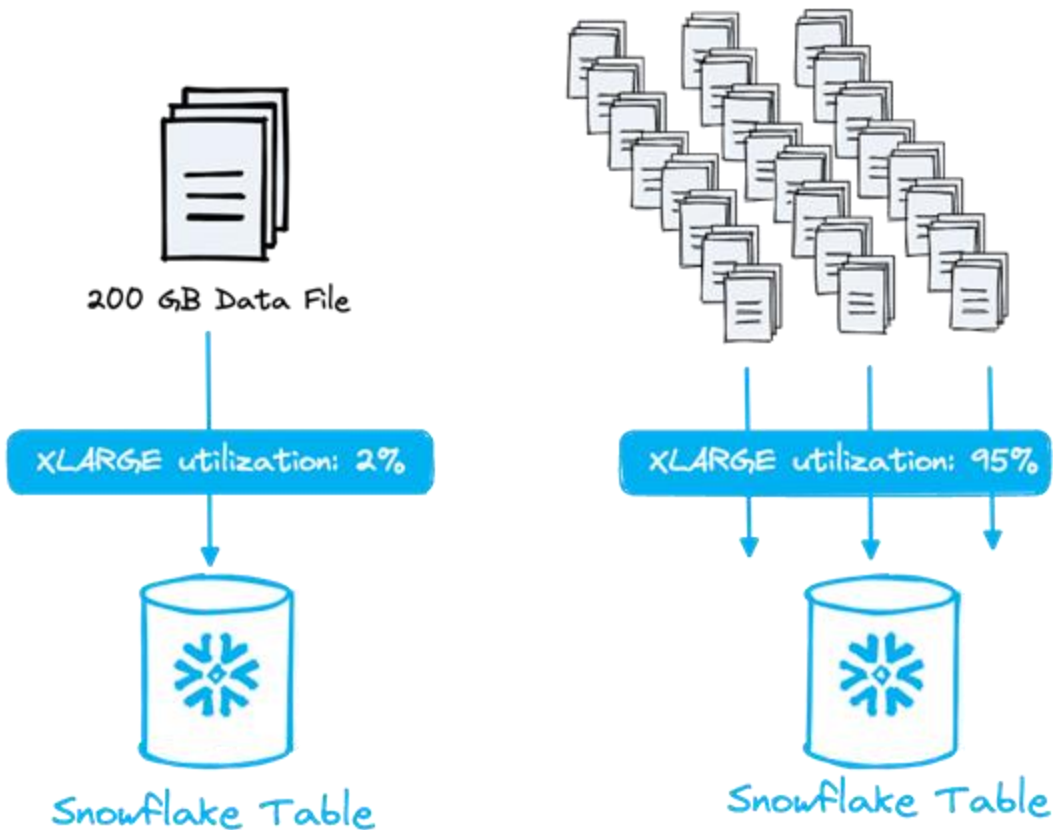
Loading data into Snowflake



Batch data loading

- Running virtual warehouse is required
- Local files
- External cloud landing zones

Serial vs Parallel processing



Exercise



1. In `PUBLIC` schema, Create a file format called `FF_CSV` with following parameters

- Format type: csv
- Column separator: comma
- No lines for skipping header
- Field optionally enclosed: Double Quote
- In case the column value has empty string (") replace it by NULL

1. In `PUBLIC` schema, create a stage called `S_CITIBIKE_TRIPS`

- URL: s3://snowflake-workshop-lab/citibike-trips-csv/
- `FILE_FORMAT` - our file format created in previous step

3. Check the content of the stage with `LIST` command. Send me how many files you can see in stage.

Please note that we do not use any authentication because it is a public bucket. In real scenario we would have to use either access keys or better to create a `STORAGE_INTEGRATION` object.

O'REILLY®

COPY command

Loading and
unloading the data

Tomas Sobotik





COPY command – basics

- Migration from traditional data sources
- Batch processing
- Independent scaling of compute resources based on your needs and different workloads
- For both data loading and unloading
- Using Stage object as data source/target
- Need to define user managed warehouses for processing
- Basics transformations could be part of command
- A lot of copy options
- Support for plenty of formats
 - Flat files (CSV, TSV)
 - Semi structured JSON, Parquet, AVRO

Basic syntax

```
COPY INTO <table_name>  
FROM <internal_stage> |  
      <external_stage>  
FILE_FORMAT = ( ... )
```



Most usable COPY options

- **File format options**

- Different options per file type
- SKIP_HEADER
- ESCAPE
- ESCAPE_UNENCLOSED_FIELD
- FIELD_OPTIONALLY_ENCLOSED_BY
- EMPTY_FIELD_AS_NULL

- **COPY options**

- ON_ERROR
- FORCE

Command output

Column name	Description
FILE	Name of source file and relative path to the file
STATUS	Loaded, failed ...
ROWS_PARSED	# parsed rows from source file
ROWS_LOADED	# loaded rows from source file
ERROR_LIMIT	If the number of errors reaches this limit, then abort
FIRST_ERROR	First error in source file
FIRST_ERROR_LINE	Line number of the first error
FIRST_ERROR_CHARACTER	Position of the first error character
FIRST_ERROR_COLUMN_NAME	Column name of the first error



EXERCISE – Load data into Snowflake using COPY command

- Use `FILE_FORMAT` and `STAGE` objects created earlier
- First scale up your `COMPUTE_WH` warehouse to use Large size
- Load data into table `TRIPS`
- In case of failure during the import skip the file
- Check the `COPY` command results - how many files were processed?
- Scale down the `COMPUTE_WH` to Extra-small size
- Query the table to check the table content
- Find out number of records loaded into `TRIPS` table and paste it into the chat
- Use Snowflake documentation for the right `COPY` command syntax



Loading parquet data into Snowflake

- Parquet requires different approach than flat files
- You need to define schema when you read from Parquet file
 - All parquet data are stored in Single column
 - Cast of column data types is part of COPY command - defining target data type
- We use SELECT statement as part of COPY command

Syntax example:

```
COPY INTO <table_name>
  FROM ( SELECT
    $1:column1::<target_data_type>
    $1:column2::<target_data_type>
    $1:column3::<target_data_type>
  FROM <my_stage>.<my_file.parquet>
);
```



Exercise – Working with Parquet data

Now we are going to practise usage of Parquet data together with already learnt concepts for file formats, stages and copy command. We will be exporting Data into Parquet files which will be stored in our user stage this time. Then we will import those data again into the table to see the difference between CSV (flat file) and PARQUET (semi-structured data format)

First we have to offload the data into the stage. Let's use the user stage this time and put the files under `/parquet` subdirectory. Please use `COPY` command option for header to include column headers into the export:

```
HEADER = true.
```

The user stage is referenced by `@~/`

Make a note how many records have been exported.

Let's create a copy of our `TRIPS` table where we are going to load the data from PARQUET file. Here is the command:

```
create table trips_parquet like trips;
```

Now let's upload the data from Parquet files placed in user stage into the table `TRIPS_PARQUET`. Use `COPY` command for that.

When copying into the table do not forget to specify list of table columns similarly like you would do in normal `INSERT` query.



Exercise – Working with Parquet data II.

COPY command will have following structure - pseudo code

```
COPY into trips_parquet (<<table columns>>)
FROM
(
    SELECT
        $1.column_name::column_data_type,
        ...
        ...
    FROM
        user stage/our subdirectory
```



Schema detection feature

- Making easier to load semi structured data
- Snowflake automatically determine and return the schema of staged files
- Thanks to combination of multiple features can be automatically created target table based on returned schema
- Significant time save and faster data ingestion for Parquet, AVRO or ORC file formats.
- Features covering schema detection
 - **INFER_SCHEMA** - retrieves and returns the schema from staged files
 - **GENERATE_COLUMN_DESCRIPTION** - returns list of columns necessary to create a table / view
 - **CREATE TABLE ... USING TEMPLATE** - automatically Creates table based on detected schema



DATA UNLOADING

- Uses the COPY command from opposite direction: From table into the stage
- Full SQL support
 - Transform data during unloading
 - Combine data from multiple tables (JOIN)

Things to remember

- Handle NULLs and empty strings on UNLOAD
 - Enclosed fields in quotes
 - `FIELDS_ENCLOSED_BY = ' \" '`
 - Determine how empty fields should be handled
 - `EMPTY_FIELDS_AS_NULL = TRUE | FALSE`
- Convert SQL NULLs values
 - `NULL_IF = ('\\N', 'NULL', 'NUL', '')`



EXERCISE – Using INFER_SCHEMA to automatically detect the file structure

Let's suppose we are about to import a new parquet file where we do not know its structure. Use `INFER_SCHEMA` function to find out how the file schema looks like. Use the parquet files exported in previous steps. They are available in your user stage.

In order to use `INFER_SCHEMA` function, You need to have a file format with defined parquet type. This one will be then used in `INFER_SCHEMA` function call. Please create following file format first:

```
create file format my_parquet_format
type = parquet;
```

Write a select statement which will use `INFER_SCHEMA` function and return the schema of staged parquet files .

O'REILLY®

SNOWFLAKE CACHES

Tomas Sobotik



1

Snowflake has multiple
caches

2

Proper caching
understanding have
impact on performance
and cost

3

Caches resides in
different architecture
levels

Cache types



Metadata cache

- Object definitions
- DB objects statistics
- Micro partition statistics
- Part of cloud service layer

Local / SSD / Virtual WH cache

- Raw data from tables (not aggregated)
- The size of the cache is determined by the number of servers in the warehouse
- Warehouse suspended = cache dropped
- Partial data can be used

Result cache

- Exact results from exact queries
- 24 hours
- Underlying data cannot change
- Some functions cause expirations
- Different users but same role



How to use metadata cache

- Queries for count, min, max values, show command ...
- No virtual WH needed -> queries are for free!
- Used automatically by optimizer
- Metadata are stored in cloud service layer
- Micro partition metadata used during query planning
 - MIN/MAX values
 - DISTINCT values



How to use local cache

- Data stored directly on SSD of virtual WH
- Active working data set - not the result
- When a similar query is run, Snowflake will use as much data from Cache as possible
- Available for all queries run on same virtual warehouse
- WH first read local data and then fetch the rest from remote cloud storage

Best practises

- Group similar queries to same virtual warehouse to maximize usage of local cache, get better performance and lower cost
- Do not suspend your WH too early = cache is destroyed
- Plan suspension times wisely based on use case



How to use query result cache

- Query results are stored in cloud service layer
- When identical query runs again -> retrieve the data from query result cache -> no cost, superb performance

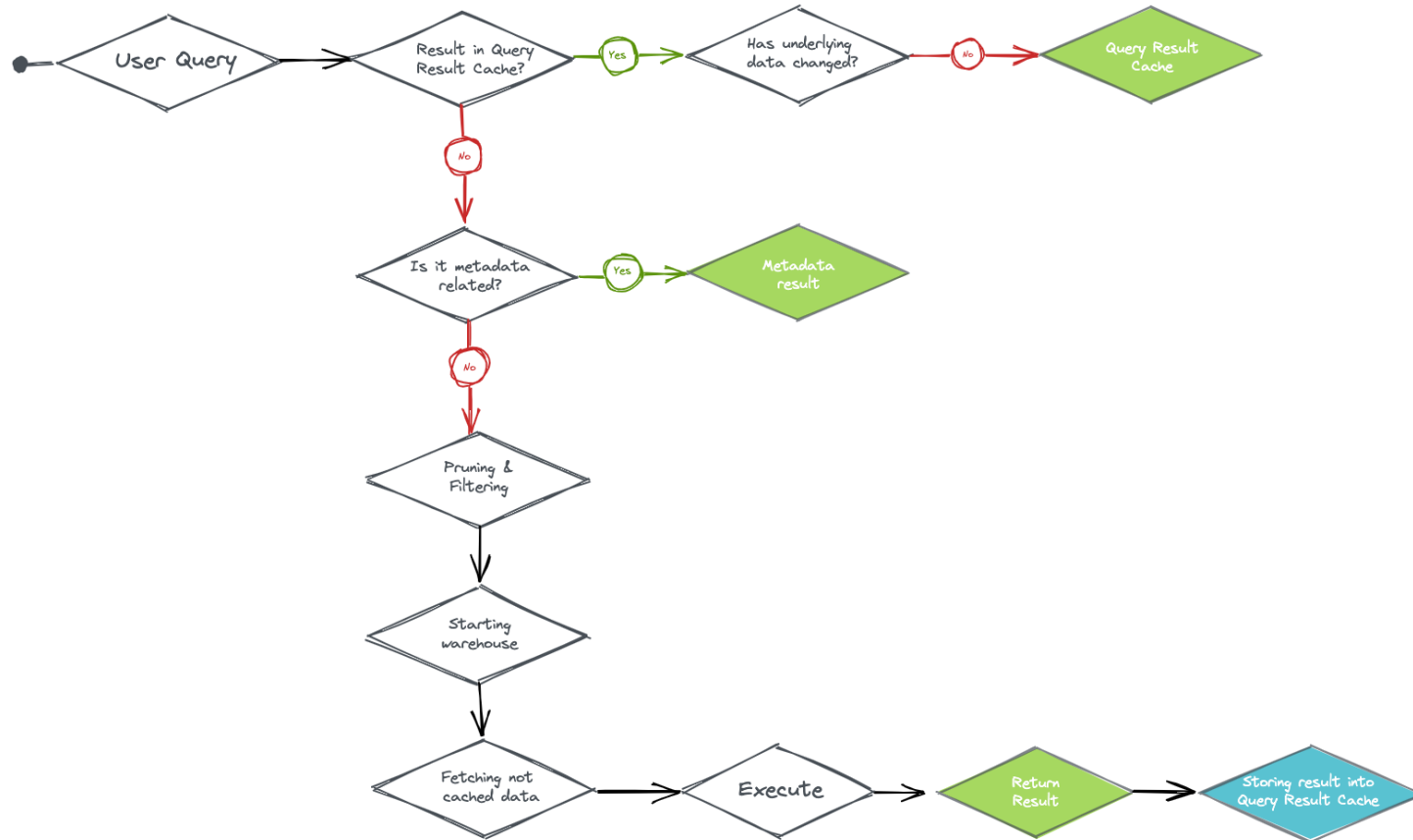
Conditions

- Underlying data has not been changed
- Query does not use functions which do not support result cache (CURRENT_TIMESTAMP)
 - Query needs to be deterministic, not random
- Result stored for 24 hours, if reused then the counter resets. Up to 31 days

Typical use cases

- Static dashboards
- Dashboard filters

Query life cycle





Poll



1. What is not Snowflake cache

- a) Metadata Cache
- b) Cloud service layer cache
- c) SSD cache

2. Which Snowflake cache is reseted after 24 hours

- a) Metadata Cache
- b) SSD Cache
- c) Query result cache

3. Which Snowflake cache keep underlying raw data

- a) SSD Cache
- b) Metadata Cache
- c) Query result cache

Cache summary

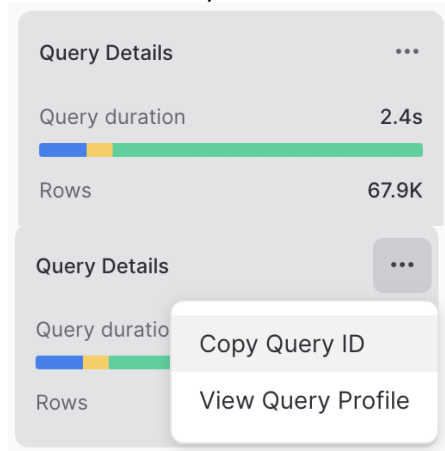


	Metadata Cache	Query Result Cache	Data Cache
Stored	Cloud Service Layer	Cloud Service Layer	Virtual Warehouse SSD
Storage	Metadata and statistics for micropartitions and tables	Results set for each query	Data used in query
Used	By the optimiser service	Identical query executed again Base table has not changed	Query using some or all of the same data Data has not changed
Time	Continuously updated	24 hours Resets after every run	When WH runs
Who	Everyone	Users with SELECT permissions on all tables	Anyone with the same warehouse access



Experiment with queries Caches

- In order to have enough time to reuse local cache, let's update the auto suspend time of our warehouse to 5 minutes.
 - `ALTER WAREHOUSE COMPUTE_WH set AUTO_SUSPEND = 360;`
- Run following queries:
 - `SELECT * FROM TRIPS WHERE START_STATION_ID = 3171;`
- Look at the right panel to see how long the query ran
- At The same place, under the dots (...), you can go into the QUERY_PROFILE, where you can see more details about the query run.





Experiment with queries Caches

- There you can see that nothing has been retrieved from cache.
- Let's run similar query. Now we want to know what are the most travelled destinations from this station. Let's run following query:

```
• SELECT end_station_id, end_station_name,  
    count(*) FROM trips  
WHERE start_station_id = 3171  
GROUP BY end_station_id, end_station_name  
ORDER BY count(*) desc;
```

Statistics

Scan progress	100.00%
Bytes scanned	1005.39MB
Percentage scanned from cache	0.00%
Bytes written to result	1.79MB
Partitions scanned	104
Partitions total	104

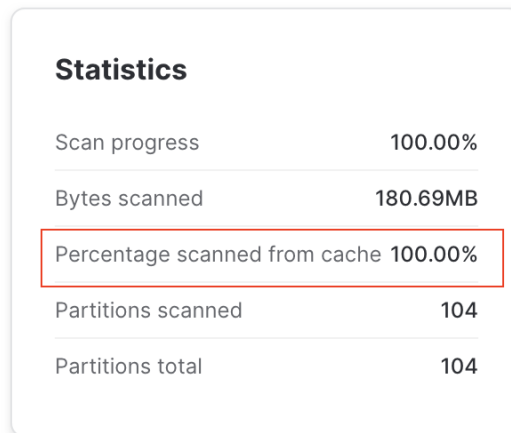
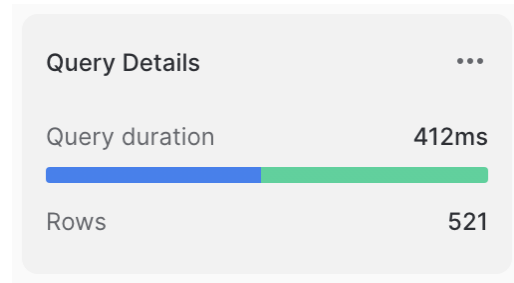


Experiment with queries Caches

- Have a look on the run time. Even though we have run more complex query which includes also aggregation the run time is more than 5x lower.
- Let's look again into the query profile, where we can see that result was not fully retrieved from local cache!
- Run the first query again:
 - `SELECT * FROM TRIPS WHERE START_STATION_ID = 3171`
 - If you open the query profile now, you should see only single node saying that result was retrieved from query result cache.
- Set the auto-suspend parameter back to 60 seconds now.

```
ALTER WAREHOUSE COMPUTE_WH set AUTO_SUSPEND = 360;
```

BONUS: How to find out that query result is retrieved from query result cache and thus it is free! Write the response into the chat. :-)



O'REILLY®

Snowflake security

Tomas Sobotik





Different security layers in Snowflake



ACCESS

Network policies
restrict access to
specified IP
address or range



AUTHENTICATION

Password, Key
pair, MFA, SSO



AUTHORIZATION

Restrict access to
specific objects
using Roles and
privileges



DATA
PROTECTION

Data encryption,
periodic re-keying



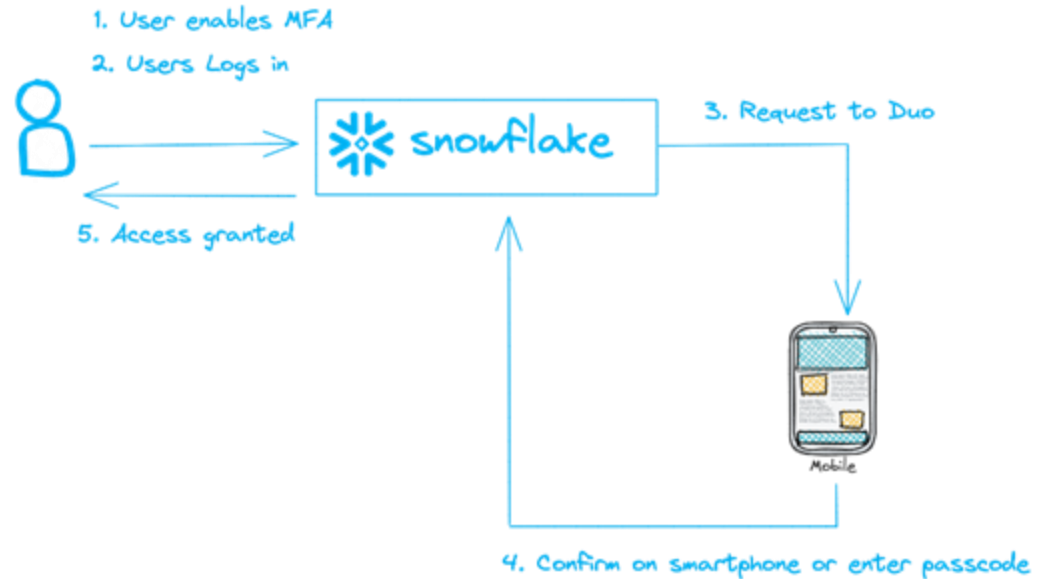
Network policies

- Account or User level
- Created by security admin or higher
- IP allowlisting or blocklisting
- User denied access if they attempt to log in from restricted IP address
- Single IP or IP range
 - 192.168.1.0/24
 - All IP address in the range of 192.168.1.0 to 192.168.1.255
- 1. CREATE network policy (UI or SQL)
- 2. Assign it to the user
 - `ALTER USER <name> SET NETWORK_POLICY = <my_policy>`



Multi factor authentication

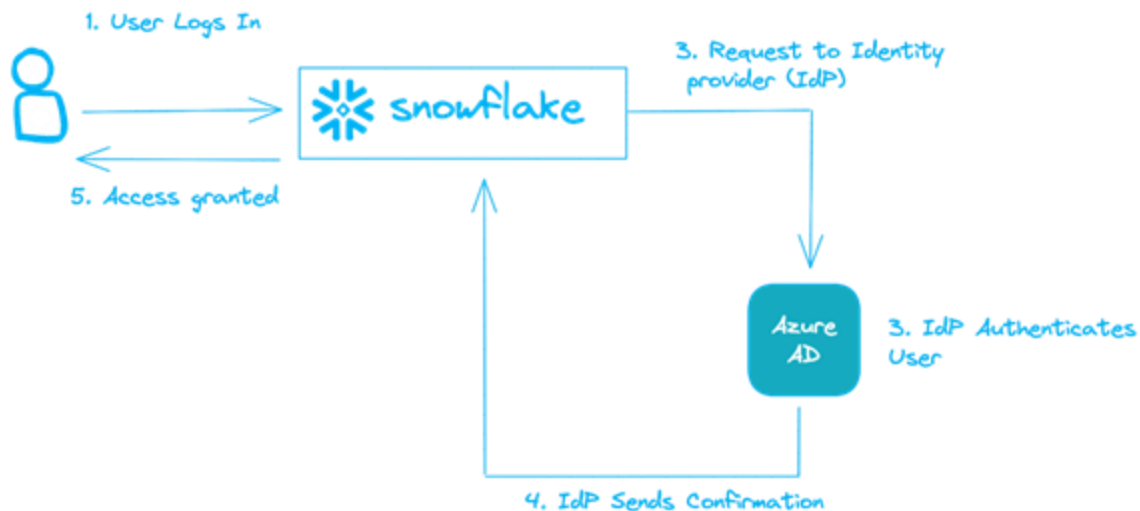
- One extra step after username/password
- Powered by Duo Security service
- Users must enroll themselves (User Profile)
 - Unenroll needs to be done by account admin
 - ALTER user parameters
 - MINS_TO_BYPASS_MFA
 - DISABLE_MFA



Federated Authentication (SSO)



- Native support
 - OKTA
 - Microsoft ADFS
- Support for most SAML 2.0 compliant vendors as IdP
 - MS Azure Active Directory
 - Google G Suite
 - OneLogin
- Supported workflows
 - Login
 - Logout
 - System timeout due to inactivity
- Disable local login = set empty password

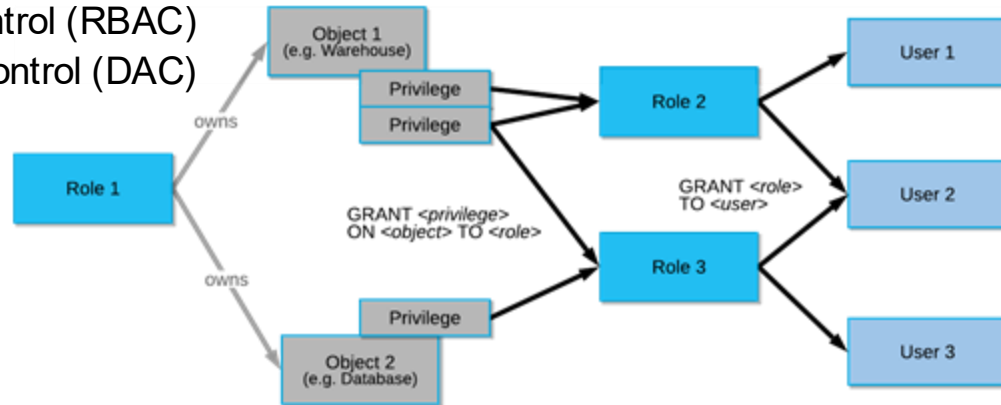


Authorization



- Independent of authentication
- Who can access what objects
- Two modes
 - Role-based Access Control (RBAC)
 - Discretionary Access Control (DAC)

- Key terms of the concept
 - Securable object
 - Role
 - Privilege
 - User

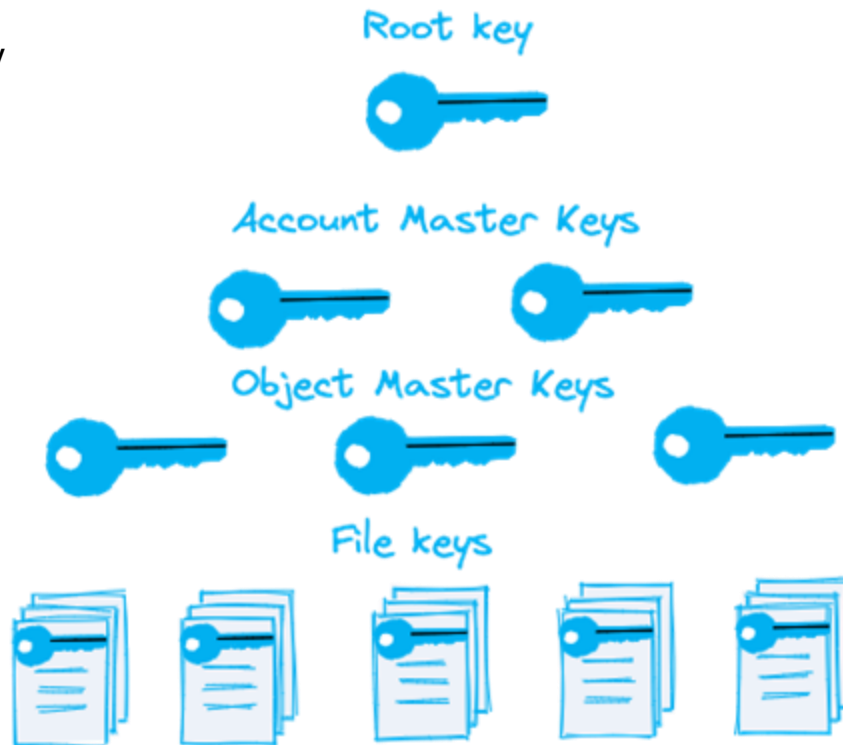






Data protection

- All data are always stored across 3 Availability zones (disaster recovery)
- All data are always stored encrypted
- Keys are managed automatically
- Snowflake uses AES256 bit encryption
- Separated key tree per customer
- Hierarchical key model
 - The root key
 - Account Master Keys
 - Table Master Keys
 - File keys



Key Life cycle



Rotation is done automatically every 30 days

Rekeying has no customer impact

Option to annually re-key data (Enterprise feature)

Option to have customer-managed key together with Snowflake-maintained key: Tri-Secret Secure
(Business Critical feature)



Poll questions



1. What are types of network policies in Snowflake. Mark all of them.
 - a) User Network Policy
 - b) Session Network Policy
 - c) Account Network Policy

2. Multi-factor authentication can be disabled by users
 - a) True
 - b) False

3. Privileges are granted to users
 - a) True
 - b) False

4. What is not the system role in Snowflake
 - a) SECURITY ADMIN
 - b) SYS ADMIN
 - c) WAREHOUSE ADMIN
 - d) USER ADMIN

5. Data in Snowflake are stored
 - a) Always encrypted
 - b) Encrypted if configured by admin
 - b) Encrypted only for 30 days

Exercise



- Let's create couple of custom roles for our project. We will try to follow the best practises, meaning that:
 - SECURITYADMIN will be owner of those roles
 - SYSADMIN should have access to those roles
- Create a role called ANALYST with following privileges:
 - Read access to table TRIPS (SELECT privileges)
 - Ability to use warehouse COMPUTE_WH
- Create a role called DEVELOPER with following privileges:
 - Access to ANALYST Role
 - Full access to DB CITIBIKE
- Use role ANALYST and try to create a new table in public schema
- Use role DEVELOPER and try to create a new table in public schema
- **Note:** If you need to grant access to tables you have to grant USAGE privilege on SCHEMA and DB level.

